

Exactly — that's a materially different architecture.

The constraint layer is **not an overlay**. It's an emergent property of the matrix.

You've basically got a multidimensional scheduling puzzle where every story occupies cells in a constrained space:

- **R / X = swimlane** — resource, person, team, machine, client, etc.
- **C / Y = channel** — state, work type, process stage, queue, responsibility domain.
- **T = time** — discretized at **15-minute minimum resolution**.
- **D / Z = drill level / organizational hierarchy / sub-board context** — the dimension that lets you move from org → department → team → individual, or project → epic → story.

And the Sudoku analogy is better than “constraint management,” because the system doesn't need a separate engine saying *here are the constraints*. The constraints fall out naturally from invalid or competing occupancy.

If:

```
R = Alice  
C = Development  
T = 10:00-11:00
```

is already occupied at capacity, then another story trying to occupy the same effective cell creates a collision.

Likewise, if a downstream story requires:

```
Story B.start >= Story A.finish
```

and Story A moves rightward on T, the available solution space for Story B changes automatically.

So the system really becomes something like:

```
M[R, C, T, D] -> occupancy / story / capacity state
```

with rules governing legal occupancy.

Then friction isn't something you *calculate afterward*.

It becomes visible because portions of the matrix become:

```
valid
under-capacity
at-capacity
over-capacity
blocked
dependency-invalid
unavailable
```

That is a much stronger model.

And I agree completely on **15 minutes**. Below that, you're no longer improving operational visibility; you're turning the interface into a timesheet and increasing cognitive load. Fifteen minutes is fine enough to represent meaningful scheduling changes while still allowing the human brain to scan the geometry.

The really powerful part is that zooming T doesn't have to change the underlying resolution.

Internally:

```
1 quantum = 15 minutes
```

But visually:

```
zoom in   -> 15m / 30m
normal    -> hour
zoom out  -> day
far out   -> week / month / quarter
```

So October 14 can contain 96 addressable time cells while the executive view sees only a single aggregated column.

That also means vacations aren't "constraints added to the system."

A vacation is simply occupancy/unavailability:

```
M[Alice, *, Oct-12 08:00..Oct-19 17:00, Team-A] = unavailable
```

Now every story whose solution requires Alice during that interval loses those cells from its legal placement space.

That's straight-up **constraint propagation**.

And *that* is where your Sudoku comparison really clicks.

Move one story, and the system doesn't just update that card. It changes the available solution domain for every related story.

Conceptually:

```
move A
  ↓
recalculate legal cells for dependents
  ↓
capacity changes
  ↓
future collisions become visible
  ↓
propagate through dependency graph
```

You aren't really designing a fancy Kanban board anymore.

You're designing a **visual constraint-satisfaction system whose primary UI happens to look like Scrum/Kanban**.

That framing also changes what I'd look for in an open-source base. We don't primarily need "the best project-management tool." We need the cleanest combination of **drag/drop + hierarchical data + extensible rendering + time virtualization**, because the real intellectual property is the matrix renderer and its propagation rules.